UDESC



Rust

Cristiano Damiani Vasconcellos
cristiano.vasconcellos@udesc.br

UDESC

struct
(produto)

```
#[derive(Debug, Clone, Copy)]
  struct Ponto {
  x: u32, y: u32
}


fn main() {
  let mut p1 = Ponto {x:1, y:10};
  p1.x = 200;
  println!("{p1:?}");
  let p2 = p1;
  println!("{p2:?}");
  println!("{p1:?}");
  let p3 = &p1;
  println!("{}", p3.y);
  p3.y = 12; //Erro
  println!("{p3:?}");
}
```

UDESC

enum
(união disjunta)

```
#[derive(Debug)]
enum Semana {
  Dom, Seg, Ter, Qua, Qui, Sex, Sab
}
use Semana::{*};

fn prox_dia_util(d:Semana) -> Semana {
  match d {
    Seg => Ter,
    Ter => Qua,
    Qua => Qui,
    Qui => Sex,
    _   => Seg
  }
}
fn main (){
  let hoje = Semana::Qui;
  println!("{hoje:?}");
  println!("{:?}", prox_dia_util(hoje));
}
```

UDESC

enum
(união disjunta)

```
#[derive(Debug)]
enum Semana {
  Dom, Seg, Ter, Qua, Qui, Sex, Sab
}
use Semana::{*};

fn prox_dia_util(d:Semana) -> Semana {
  match d {
    Seg => return Ter,
    Ter => return Qua,
    Qua => return Qui,
    Qui => return Sex,
    _   => return Seg
  }
}
fn main (){
  let hoje = Semana::Qui;
  println!("{hoje:?}");
  println!("{:?}", prox_dia_util(hoje));
}
```

UDESC

enum com parâmetros
(união disjunta)

```
use std::f64::consts::PI;
use Forma::{*};

#[derive(Debug, Clone, Copy)]
enum Forma {
  Circulo   {raio:f64},
  Retangulo {alt:f64, compr:f64}
}
fn area(f:Forma) -> f64 {
  match f {
    Circulo   {raio: r} => r * r * PI,
    Retangulo {alt: a, compr: c} => a * c
  }
}
fn main (){
  let c = Circulo {raio:2.0};
  println!("{c:?}");
  println!("{}", area(c));
  println!("{c:?}");
}
```

UDESC

enum com parâmetros
(união disjunta)

```
use std::f64::consts::PI;
use Forma::{*};

#[derive(Debug)]
enum Forma {
  Circulo   {raio:f64},
  Retangulo {alt:f64, compr:f64}
}
fn area(f:&Forma) -> f64 {
  match *f {
    Circulo   {raio: r} => r * r * PI,
    Retangulo {alt: a, compr: c} => a * c
  }
}
fn main (){
  let c = Circulo {raio:2.0};
  println!("{c:?}");
  println!("{}", area(&c));
  println!("{c:?}");
}
```

UDESC

Funções Associadas

```
use std::f64::consts::PI;
use Forma::{*};
#[derive(Debug)]
enum Forma {
  Circulo   {raio:f64},
  Retangulo {alt:f64, compr:f64}
}
impl Forma {
   fn area(f:&Forma) -> f64 {
     match *f {
       Circulo   {raio: r} => r * r * PI,
       Retangulo {alt: a, compr: c} => a * c
     }
   }
}
fn main (){
  let r = Retangulo {alt:10.0, compr:5.0};
  println!("{r:?}");
  println!("{}", area(&r));
}
```

UDESC

Métodos

```
use std::f64::consts::PI;
use Forma::{*};

#[derive(Debug)]
enum Forma {
  Circulo   {raio:f64},
  Retangulo {alt:f64, compr:f64}
}
fn area(self) -> f64 {
  match self {
    Circulo   {raio: r} => r * r * PI,
    Retangulo {alt: a, compr: c} => a * c
  }
}
fn main (){
  let r = Retangulo {alt:10.0, compr:5.0};
  println!("{}", r.area()); // Propriedade é transferida
  println!("{r:?}"); // Erro
}
```

UDESC

Métodos

```
use std::f64::consts::PI;
use Forma::{*};

#[derive(Debug)]
enum Forma {
  Circulo   {raio:f64},
  Retangulo {alt:f64, compr:f64}
}
fn area(&self) -> f64 {
  match *self {
    Circulo   {raio: r} => r * r * PI,
    Retangulo {alt: a, compr: c} => a * c
  }
}
fn main (){
  let r = Retangulo {alt:10.0, compr:5.0};
  println!("{r:?}");
  println!("{}", r.area());
}
```



Métodos

```
use std::f64::consts::PI;
use Forma::{*};

#[derive(Debug)]
enum Forma {
  Circulo   {raio:f64},
  Retangulo {alt:f64, compr:f64}
}
fn area(&self) -> f64 {
  match self {
    Circulo   {raio: r} => r * r * PI,
    Retangulo {alt: a, compr: c} => a * c
  }
}
fn main (){
  let r = Retangulo {alt:10.0, compr:5.0};
  println!("{r:?}");
  println!("{}", r.area());
}
```



&f64



autoderef

Métodos

```
use Forma::{*};
#[derive(Debug)]
enum Forma {
  Circulo {raio:f64},
  Retangulo {alt:f64, compr:f64}
}
impl Forma {
  fn escala(&mut self, e:f64){
    match self {
    Circulo {raio: r} => {*r = *r * e;}
    Retangulo {alt: a, compr: c} => {*a = *a * e; *c = *c * e;}
    }
  }
}
fn main (){
  let mut r = Retangulo {alt:10.0, compr:5.0};
  println!("{r:?}");
  r.escala(2.0);
  println!("{r:?}");
}
```



&mut f64



Tem que ser explícito

UDESC

Tuplas

Tuplas são coleções de diferentes tipos de um tamanho fixo, por exemplo, duplas, triplas, quadruplas, etc. Os elementos das tuplas podem ser acessados pelos índices de suas posições: 0, 1, 2, ...

```
fn main() {
    let mut a = (2, 'A');
    let b = (true, 2.5, 1);
    a.0 = 10;
    println!("{a:?}");
    println!("{b:?}");
}
```